

Database Authentication Password Security

From Default Authentication to Database Authentication

When the following dependency is added to a Spring Boot application, Spring Security automatically secures the application:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

By default, Spring Boot creates a user with the username `user`, prints a randomly generated password in the console, secures every endpoint, and provides form login and HTTP Basic authentication.

After successful authentication, Spring Security creates an `Authentication` object representing the currently authenticated user.

```
Authentication
-----
principal      = user
credentials    = protected/erased
authorities    = ROLE_USER
authenticated  = true
```

The important question is:

| Where does Spring Security obtain the information required to verify a user?

1. Authentication Requires a Source of Truth

Suppose a client submits:

```
username = aditya
password = secret123
```

Spring Security cannot automatically know whether these are correct or not. It needs trusted information against which it can validate the

User-submitted information

```
-----
username = aditya
password = secret123
```

Trusted information

```
-----
username = aditya
passwordHash = ...
roles = ...
accountStatus = ...
```

Authentication can therefore be understood as:

Claimed identity + credential



Validation against



Trusted user information

The trusted information can live in application memory, a relational database, MongoDB, LDAP, an external identity provider, or a separate authentication service.

In this application, the source of truth will be a MySQL database:

Spring Boot Application



MySQL Database

2. Why the Default User Is Not Enough

Spring Security's generated user is useful for learning and to practical user-management system.

A real learning platform could have thousands of users. Every own identity, password, roles, and account status. This information application restarts, server restarts, deployments, and applications. The users must therefore be stored in persistent storage such

3. Why Spring Security Does Not Directly Understand Our Database

Every application has a different database design.

One application may use a `users` table with fields such as `id`, `email`, `password`, `phone`, `created_at`, and `enabled`. Another may use an `employees` table with `employee_id`, `corporate_email`, `credential_hash`, `department`, and `active`. Other systems may call the same concept `customers`, `members`, or `accounts` and may use a completely different database technology.

Spring Security cannot require every application to use the same table name, column names, or database design. Instead, it defines common security abstractions.

One of the most important abstractions is:

`UserDetails`

`UserDetails` represents the user information Spring Security needs during username-and-password authentication.

Application-specific database model

↓

Adapter

↓

Spring Security's `UserDetails`

This keeps Spring Security independent of the application's database structure.

4. Identity and Credentials

Authentication must answer two questions:

Who are you?



Identity

Can you prove it?



Credential

For username-and-password authentication:

```
username or email → identity
password          → credential
```

Username versus email

An application does not have to use a traditional username as the login identifier. It may use a username, email address, employee ID, phone number, or any other value that uniquely identifies an account.

For example, an application can use:

```
findByEmail(email);
```

instead of:

```
findByUsername(username);
```

Spring Security APIs use historical names such as `loadUserByUsername(...)` and `getUsername()`. However, the value can still represent an email address if that is the application's login design.

| The login identifier must uniquely identify exactly one account.

The following data would make authentication ambiguous:

ID	EMAIL
1	aditya@gmail.com

The login identifier should therefore have a unique database example uses `username` to keep the terminology aligned with interfaces.

6. Designing the Authentication Database

The absolute minimum information required for username-and-password authentication is:

```
User
├── username
└── password
```

A minimal table could be:

```
users
-----
id          BIGINT PRIMARY KEY
username    VARCHAR UNIQUE
password    VARCHAR
```

In a realistic application, account status and roles are also useful.

7. Understanding `UserDetails`

A simplified view of Spring Security's `UserDetails` interface is:

```
public interface UserDetails {

    String getUsername();

    String getPassword();
```

```

    Collection<? extends GrantedAuthority> get
    boolean isAccountNonExpired();

    boolean isAccountNonLocked();

    boolean isCredentialsNonExpired();

    boolean isEnabled();
}

```

It contains three main categories of information:

1. Identity — `getUsername()`
2. Credential — `getPassword()`
3. Authorities and account status

Enabled and disabled accounts

An account may need to remain in the database even when it should no longer be allowed to authenticate.

User exists
but
User cannot authenticate

For this requirement, the user table can contain an `enabled` field:

```

users
-----
id | username | password | enabled

```

Example:

```
1 | aditya | $2a$... | true
2 | rohit | $2a$... | false
```

An enabled user may continue through authentication. A disabled user is not eligible for successful authentication.

Disabled versus locked

Disabled

→ The account is inactive.

Locked

→ The account exists, but authentication is temporarily blocked.

An account might be locked after repeated failed login attempts without being permanently disabled.

Account expiration

`isAccountNonExpired()` represents whether the account itself is still valid. For example, a temporary employee's account may be valid only until the end of a contract.

Credential expiration

`isCredentialsNonExpired()` represents whether the user's credential is still valid. A company may, for example, require a password to be changed after 90 days.

Account expired

→ The account itself is no longer valid.

Credentials expired

→ The account may be valid, but its credential must be renewed.

Spring Security provides hooks for these states, but every need a separate database column for each one.

If an application does not support account expiration, its `User` implementation can simply return `true` from `isAccountNonExpire`

8. Initial `User` Entity

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    private boolean enabled = true;
}
```

The `unique = true` constraint is important because one username must identify only one account.

Domain `User` versus Spring Security `UserDetails`

The application's `User` entity and Spring Security's `UserDetails` abstraction belong to different layers.

`Application/domain` layer

User Entity

- └ email
- └ profile
- └ createdAt
- └ password
- └ roles

↓ adapt

Spring Security layer

UserDetails

- └ username
- └ password
- └ authorities
- └ account status

The `User` entity models what the application wants to store. `UserDetails` models what Spring Security needs for authentication. They can be connected through an adapter without treating them as the same concept.

9. Introducing Roles

Authentication establishes who the user is. Authorization determines what that authenticated user is allowed to do.

A learning platform might define:

```
STUDENT → View courses
TEACHER → Create courses
ADMIN   → Delete users
```

A user may have one or more roles:

```
Aditya
└ ROLE_USER
```

└─ **ROLE_ADMIN**

Rohit

└─ **ROLE_USER**

Database structure

users

id | username | password | enabled

roles

id | name

user_roles

user_id | role_id

Example data:

users

1 | aditya | \$2a\$... | true
2 | rohit | \$2a\$... | true

roles

1 | **ROLE_USER**
2 | **ROLE_ADMIN**

user_roles

1 | 1
1 | 2
2 | 1

The join-table entries mean that Aditya has both roles, while

`ROLE_USER`.

10. JPA Entity Model for Roles

Role entity

```
@Entity
@Table(name = "roles")
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;
}
```

Updated **User** entity

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
```

```

private String password;

private boolean enabled = true;

@ManyToMany
@JoinTable(
    name = "user_roles",
    joinColumns = @JoinColumn(name = "user_id"),
    inverseJoinColumns = @JoinColumn(name = "role_id")
)
private Set<Role> roles = new HashSet<>();
}

```

The mapping is unidirectional because the authentication use case needs `user.getRoles()` but does not currently require `role.getUsers()`.

A bidirectional relationship should be added only if the application genuinely needs navigation in both directions.

11. Repository Layer

The application needs a repository to retrieve users by their login identifier:

```

public interface UserRepository
    extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);
}

```

A role repository may also be useful during registration or role management:

```

public interface RoleRepository
    extends JpaRepository<Role, Long> {
}

```

```
Optional<Role> findByName(String name);  
}
```

At this stage, Spring Security still does not directly call

```
userRepository.findByUsername(...).
```

```
Database  
  ↑  
UserRepository  
  ↑  
Missing bridge  
  ↑  
Spring Security
```

The missing bridge is `UserDetailsService`.

`DaoAuthenticationProvider` uses `UserDetailsService` to load the username, encoded password, authorities, and account-status information required for authentication.

12. Why Raw Passwords Must Never Be Stored

Consider a registration request:

```
POST /register  
Content-Type: application/json
```

```
{  
  "username": "aditya",  
  "password": "secret123"  
}
```

The simplest implementation would save the password directly:

```
User user = new User();  
user.setUsername(request.getUsername());
```

```
user.setPassword(request.getPassword());  
  
userRepository.save(user);
```

The database would then contain:

id	username	password
1	aditya	secret123

This may work functionally, but it is extremely unsafe. If the database is compromised, every stored password is immediately exposed. The damage may extend beyond this application because users frequently reuse the same password on multiple services.

| An application should never intentionally store a user's raw password.

13. Why Password Encryption Is Not the Right Solution

Encryption is reversible:

```
Plaintext  
  ↓ encryption + key  
Ciphertext  
  ↓ decryption + key  
Plaintext
```

If passwords are encrypted, the application must keep a decryption key capable of recovering every original password. If an attacker obtains both the database and the key, all passwords can be recovered.

The more fundamental question is:

| Does the application ever need to recover a user's original password?

No. During login, the application only needs to determine whether the entered password matches the password registered earlier.

Therefore, passwords need a one-way transformation rather than encryption.

14. Password Hashing

Hashing changes the security model:

```
secret123
  ↓
One-way password function
  ↓
Encoded representation
```

The encoded representation is stored in the database. There should be no operation that converts it back into the original password.

During login, the system performs verification:

```
Entered raw password
      +
Stored encoded password
      ↓
Does the entered password correspond
to this stored representation?
      ↓
true / false
```

Spring Security expresses this operation through:

```
passwordEncoder.matches(rawPassword, encodedPassword);
```

15. Why a Fast Hash Such as SHA-256 is Not Enough

A general-purpose cryptographic hash such as SHA-256 is designed for speed. Speed is valuable for many computing tasks, but it is dangerous for password storage.

If attackers steal a password-hash database, they can perform an offline guessing attack:

```
password
123456
qwerty
secret123
welcome123
...
```

For every guess, they calculate a hash and compare it with the stolen value. A very fast algorithm allows attackers to test an enormous number of guesses quickly.

Password-storage algorithms are deliberately designed to make every guess computationally expensive.

```
Fast general-purpose hash
→ Attackers can test guesses rapidly.

Adaptive password hash
→ Every guess requires more computation.
```

Spring Security supports adaptive one-way password functions such as BCrypt for this purpose.

16. BCrypt and PasswordEncoder

Spring Security provides `BCryptPasswordEncoder`, which implements the `PasswordEncoder` abstraction:

```
PasswordEncoder
    ↑
    | implements
BCryptPasswordEncoder
```

The encoder can be registered as a Spring bean:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Depending on the abstraction allows the application to change the concrete password-encoding strategy without rewriting all authentication code.

Core operations

```
String encoded = passwordEncoder.encode("secret123");
```

The result may look similar to:

```
$2a$10$...
```

The password can later be verified using:

```
passwordEncoder.matches("secret123", encoded); // true
```

An incorrect password produces:

```
passwordEncoder.matches("wrongPassword", encoded); // false
```

The application never needs to decrypt the stored value.

17. Salt and BCrypt

A stolen database can also be attacked using precomputed password dictionaries, commonly associated with rainbow-table attack

```
password    → known hash 1
123456      → known hash 2
secret123   → known hash 3
admin123    → known hash 4
```

A salt adds unique random data to the password-hashing process.

```
Aditya
secret123 + randomSaltA
          ↓ BCrypt
        encodedValueA
```

```
Rohit
secret123 + randomSaltB
          ↓ BCrypt
        encodedValueB
```

Even though both users selected the same password, their stored BCrypt values are different.

BCrypt includes the salt and algorithm parameters inside the encoded value stored in the database. A separate salt column is therefore not required when using the standard BCrypt representation.

Practical demonstration

```
System.out.println(
    passwordEncoder.encode("secret123")
);

System.out.println(
```

```
passwordEncoder.encode("secret123")  
);
```

The two results should be different:

```
$2a$10$A.....  
$2a$10$B.....
```

How does `matches()` work if every encoding is different?

`matches()` does not encode the raw password again with a new salt and compare the two strings:

```
passwordEncoder  
    .encode(rawPassword)  
    .equals(storedPassword); // Incorrect approach
```

That approach would fail because each new encoding normally uses a different random salt.

Instead, `matches()` reads the salt and other required parameters from the stored BCrypt value. It applies those parameters while verifying the submitted raw password.

```
passwordEncoder.matches(  
    rawPassword,  
    storedEncodedPassword  
);
```

Spring Security hides the algorithm-specific verification details behind the `PasswordEncoder` interface.

18. Registration Flow from First Principles

The client sends:

```
POST /register
Content-Type: application/json
```

```
{
  "username": "aditya",
  "password": "secret123"
}
```

The registration flow is:

```
RegisterRequest
├─ username = aditya
└─ password = secret123
    ↓
    PasswordEncoder
    ↓
    BCrypt
    ↓
    Encoded password
    ↓
    User entity
    ↓
    UserRepository
    ↓
    Database
```

In short:

```
Raw password
    ↓
PasswordEncoder.encode()
    ↓
BCrypt-encoded password
```



Database

Password encoding occurs before the user entity is saved must never be persisted.

19. Registration DTO

A simple request DTO can validate the incoming values:

```
public class RegisterRequest {  
  
    @NotBlank  
    private String username;  
  
    @NotBlank  
    @Size(min = 8)  
    private String password;  
}
```

`@Size(min = 8)` demonstrates basic length validation. A production password policy may require additional rules, but password validation and password encoding solve different problems:

Validation

→ **Decides** whether the submitted password is **acceptable**.

Password encoding

→ **Protects** the password representation stored in the database.

20. Registration Service

The service must check whether the username already exist password, create the entity, and then save it.

The repository can expose an existence query:

```
public interface UserRepository
    extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

    boolean existsByUsername(String username);
}
```

A minimal registration service can be written as:

```
@Service
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthService(
        UserRepository userRepository,
        PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    public User register(RegisterRequest request) {

        if (userRepository.existsByUsername(request.getUsername())) {
            throw new IllegalArgumentException(
                "Username is already registered"
            );
        }
    }
}
```

```

    User user = new User();
    user.setUsername(request.getUsername());
    user.setPassword(
        passwordEncoder.encode(request.getPassword())
    );
    user.setEnabled(true);

    return userRepository.save(user);
}

```

The critical line is:

```

user.setPassword(
    passwordEncoder.encode(request.getPassword())
);

```

The value received in the request is raw. The value saved in the database is BCrypt-encoded.

Request DTO

secret123

raw password

Database

\$2a\$10\$...

encoded password

The raw password should exist only for the short time required to process the registration request. It should not be logged, returned in the response, or stored anywhere.

21. Complete Conceptual Picture

Registration and login use the password encoder differently.

During registration

```
Raw password
  ↓
PasswordEncoder.encode()
  ↓
Encoded password
  ↓
Database
```

22. Key Takeaways

- Authentication requires trusted user information from a source of truth.
- A database provides persistent storage for users, passwords, roles, and account status.
- Spring Security does not depend directly on an application-specific database structure.
- `UserDetails` represents the user information required by Spring Security.
- `UserDetailsService` forms the bridge between Spring Security and the application's repository layer.
- The login identifier must uniquely identify one account.
- The domain `User` entity and Spring Security's `UserDetails` are separate concepts.
- Raw passwords must never be stored.
- Password encryption is not appropriate because it is reversible.
- Fast general-purpose hashes are unsuitable for password storage.
- BCrypt is an adaptive, salted, one-way password-hashing algorithm.
- BCrypt stores the salt and required parameters inside its encoded output.
- `encode()` is used before saving a new password.
- `matches()` is used to verify a raw password against the stored encoded value.

- Registration must validate input, prevent duplicate identities, verify password, and only then save the user.